

Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

Teil 0: Organisatorisches und Einleitung

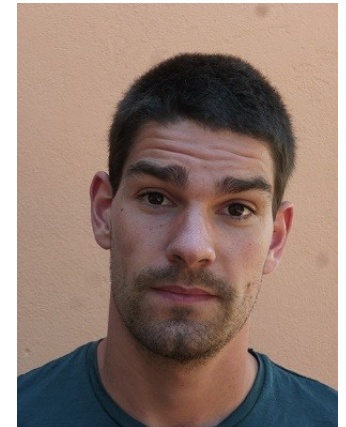
v. 17.10.2016

- **Organisatorisches**
- **Studienordnung: Einordnung / Kompetenzen / Struktur**
- **Vorlesung: Bildungsvertrag, Termine, Feedback**
- **Übung: Konzept / Termine**
- **Klausur**
- Vorstellung des Fachgebietes
- Vorlesungsinhalte

Boris Düdder



Michael Lybecait



Emily Veuhoff, Oxana Warkentin & Roland Ihle

Erlangbare Kompetenzen innerhalb der Vorlesung:

- Vermittlung eines Überblicks über das Spektrum gängiger **Konzepte zur Spezifikation und zum Testen.**
- **Einbettung dieser Spezifikationskonzepte** in die Qualitätssicherung.
- **Modellbasiertes Software-Engineering.**
- **Fortgeschrittene Spezifikations- und Verifikationstechniken** (OCL, statische Analyse)
- Software Architektur
- **Software Management** (Qualität, Projekt, Konfiguration).

Bachelor Informatik / Angewandte Informatik: **Wahlpflichtmodul.**

Teilnahmevoraussetzungen:

- **Erfolgreich abgeschlossene Module:** „Software-Technik“ (SWT)
 - „Software Praktikum“ ist nicht mehr Teilnahmevoraussetzung
- **Kenntnisse:** Objektorientierung, Programmierpraxis, Mathematische Grundlagen der Informatik

Umfang: 3 SWS (2 SWS Vorlesung, 1 SWS Übung)

4 Credits: 3 Credits Vorlesung, 1 Credit Übung

Aufwand: 120 Stunden über 15 Semesterwochen:

- 45 Stunden Präsenz ($15 \cdot (2+1)$)
- 75 Stunden Vor-/Nachbereitung und Hausübungen ($15 \cdot 5$)

Veranstaltungssprache: Deutsch.

- **Fachliche Einführung** in das Thema Softwarekonstruktion.
- **Engagierte Betreuung:**
 - Interessante Vorlesung.
 - Regelmäßige Sprechstunden.
 - Betreute Übungen.
 - Korrigierte Hausübungen.
 - Transparente Anforderungen.
 - Möglichkeiten zum direkten Feedback.
- Möglichkeit zum **Erwerb des Scheins**.

Aktives Auseinandersetzen mit den Vorlesungsinhalten:

- Aktive Teilnahme an der Vorlesung.
- Vor- und Nachbereitung der Vorlesung.
- Aktive Teilnahme an den Übungen.
- Bearbeitung der Hausübungen.

Vorlesungstermin: Mo 12:15 bis 13:45, Otto-Hahn-Str. 14 – E23

Abstimmung: PAUSE ?

Aktuelle Informationen zur Vorlesung (**bitte regelmäßig beachten wegen möglicher Vorlesungsausfälle o.ä.**):

<http://ls14-www.cs.tu-dortmund.de/swk>

Zusätzlich bei Moodle2 anmelden (Unimail-Account) und in Kurs “Softwarekonstruktion WS16/17, LSF, 040211” einschreiben.

Link: <https://moodle2.tu-dortmund.de/course/view.php?id=988>

Vorlesungsfolien werden auf o.g. Webseite zur Verfügung gestellt (planmäßig spätestens 18:00 Uhr am Vortag der Vorlesung).

Enthalten Diskussionsfolien; zugehörige Antwortfolien werden erst nach der Vorlesung online gestellt.

Vollständigkeit der Folien (Klausurvorbereitung) vs. „Textwand“:
Teilweise **Notizfolien** und **Anhänge**:

- Inhalt in Vorlesung nur mündlich wiedergegeben
- Textversion nützlich z.B. für Klausurvorbereitung
- Fragen zur Selbstkontrolle

Termine (**14-tägig**):

- Dienstags, 10:15 - 11:45, OH14/304 Gruppen 1 und 9
- Mittwochs, 12:15 - 13:45, OH14/304 Gruppen 2 und 10
- Mittwochs, 14:15 - 15:45, OH14/304 Gruppen 3 und 11
- Mittwochs, 12:15 - 13:45, OH12/1.054 Gruppen 4 und 12
- Mittwochs, 14:15 - 15:45, OH12/1.054 Gruppen 5 und 13
- Mittwochs, 12:15 - 13:45, OH12/2.063 Gruppen 6 und 14
- Mittwochs, 14:15 - 15:45, OH12/2.063 Gruppen 7 und 15
- Donnerstags, **14:15 - 15:45**, OH14/304 Gruppen 8 und 16

Ausgabe 1. Übungsblatt: 24.10.2016

Start der Übungen: KW 43 (ab dem 25.10.2016),

Übungsblätter zu Terminen mitbringen (zur Bearbeitung der Präsenzübungen)!

Anmeldung:

- Via AsSESS
- <http://ess.cs.uni-dortmund.de/ASSESS/index.php?do=lecturelist>
- Anmeldung bereits möglich.
- **Anmeldung bis 19.10.2016, 14:00 Uhr.**
- Verteilung mittels Algorithmus (Solver).
(nicht priorisiert nach zeitlichem Eingang der Anmeldung)
- **Bekanntgabe der Verteilung am 20.10.2016.**

- Ablauf der Übungen und Erwerb der Studienleistung
 - siehe Webseite zur Vorlesung und Hinweise auf 1. Übungsblatt

LS 14 hat für SWK **Web-basiertes System für automatische Bewertung von Übungsaufgaben** entwickelt
(teilweise Finanzierung durch „QVM-Mittel“- **DANKE !**).

Zwei Einsatzszenarien:

- Einsatz im Rahmen von **2 Übungsblättern**: Erlaubt zusätzliches Angebot dieser Übungsblätter mit kurzfristigem Feedback durch automatisches System (sonst aus Ressourcengründen nicht realisierbar).
- Zusätzliches Angebot für freiwillige Bearbeitung von Übungsaufgaben, z.B. **Klausurvorbereitung**.

Ziel: **Diskussion** der Studierenden untereinander.

Zusätzliche Kommunikation mit Veranstaltern für Fragen zum Inhalt:

- **Keine garantierten Antwortzeiten**
- Für Dringendes oder Vertrauliches: **Mail** oder **Sprechstunde**

Organisatorische + inhaltliche FAQ

- Für Fragen von Studierenden, die auch für andere interessant sein könnten.

Moderation durch Veranstalter.

Bachelor Informatik / Angewandte Informatik:

- **Studienleistung:** Übungsschein
 - **Modulprüfung:** Klausur (bei bestandener Studienleistung)
- => **Bearbeitung der Übungsaufgaben ist Voraussetzung** zur Teilnahme an der Modulprüfung.

Diplom:

- **Prüfungsleistung:** Klausur

Prüfung: Klausur

- schriftlich
- 60 Minuten

Klausurtermine:

- **Donnerstag, 23. Februar 2017** von 17:00-18:00 Uhr
in Audimax und Mathe E29.
- **Dienstag, 21.03.2017** von 14:00-15:00 Uhr
in SRG 1 und OH14 E23.

Wir **bitten um vorlesungsbegleitendes Feedback**, um Verbesserungen semesterbegleitend durchführen zu können.

Übliche Kontaktmöglichkeiten:

- Nach der Vorlesung
- E-mail `boris.duedder @ cs.tu-dortmund.de`
- Tel.: 0231 755-7935
- Sprechstunde: Di 14-15 Uhr (bitte vorher per Email anmelden)

Darüber hinaus: interne Umfragen für vorlesungsbegleitendes Feedback.

Vorlesungsseite (bitte regelmäßig beachten):

<http://ls14-www.cs.tu-dortmund.de/swk>

Ansprechpartner (WWW: verlinkt von o.g. Webseite):

- Vorlesung:
 - Boris Düdder
- Übung:
 - Michael Lybecait
 - Oxana Warkentin
 - Roland Ihle
 - Emily Veuhoff
- Moodle: <https://moodle2.tu-dortmund.de/course/view.php?id=988>
- Übungsanmeldung: <http://ess.cs.uni-dortmund.de/ASSESS>

- Organisatorisches
- **Vorstellung des Fachgebietes**
- **Forschung**
- **Abschlussarbeiten, Hiwi-Jobs, weiteres Lehrangebot**
- Vorlesungsinhalte

- Automatische Programmsynthese
- Machine Learning von Programmen (Fehlervorhersage)
- Algebraische Programmtransformationen
- Enterprise- und Softwarearchitektur
- Typtheorie / Theorie der Programmiersprachen
- Computational Logic (u.a. Theorembeweiser)
- Software Produktlinien

WiSe 2016/17:

- Vorlesung “Software-Konstruktion” (Bachelor-Wahlpflicht)
- **Vorlesung „Sicherheit: Fragen und Lösungsansätze“ (Bachelor-Wahlvorlesung)**
- **Vorlesung „Projektmanagement“ (Master-Pflicht)**

SoSe 2017 (unter Vorbehalt):

- **Proseminar “Softwareengineering im Cloud Computing“**
- **Vorlesung “Methodische Grundlagen des Software Engineering” (Master-Basismodul Software)**

Forschungsbereich Master: Software, Sicherheit und Verifikation

Schwerpunktgebiete Diplom: Sicherheit und Verifikation, Software-Konstruktion

Bachelorarbeiten:

- **Modelchecking eines Materialflusssystems mittels SMT Z3 Constraint Solving**
- **Kompositionales Typmodell von Softwarelizenzen für die automatische Lösung von Lizenzkonfigurationen**

Master-/Diplomarbeiten:

- **Synthese von VHDL-Spezifikationen für Hardware mittels Inhabitation von ClaSH Programmen**
- **Synthese von funktionalen Programmen für Quantencomputer**
- **Synthese von numerischen Programmen**
- **Vergleich verschiedener DSL Ansätze zur Programmgenerierung**

Aktuelle Informationen unter:

<http://ls14-www.cs.tu-dortmund.de/cms/de/Lehre/Abschlussarbeiten/index.html>

Bei Interesse **bitte bei mir melden!**

Wir suchen immer nach engagierten Studentischen Hilfskräften zur:

- Unterstützung der Forschung im Lehrstuhl 14 – Software Engineering
 - Graduiertenkolleg 2193
 - Nationales Leistungszentrum: „Logistik und IT“
- Unterstützung von Lehrveranstaltungen am LS14
- Projektmitarbeit im Fraunhofer Institut für Software- und Systemtechnik (ISST)

Bei Interesse melden Sie sich bitte bei mir oder meinen Kollegen am Lehrstuhl 14.

- Organisatorisches
- Vorstellung des Fachgebietes
- **Vorlesungsinhalte**

Software Engineering is...

... the establishment and use of **sound engineering principles** in order to obtain **economically** software that is **reliable** and works **efficiently** on real machines. [NR68]

... the **systematic** approach to the **development, operation, maintenance, and retirement of software.**“ [IEEE83]

Entwicklungsprozesse der Software-Industrie bei weiten nicht so planbar, zuverlässig, effektiv, effizient und flexibel wie die „althergebrachter“ Industrien:

- **Keine zuverlässige Herstellung** von Software im industriellen Maßstab.
- Kosten- und Terminüberschreitungen.
- Bei Auslieferung: **ungenügende Softwarereife.**
- **Keine Produktivitätskontrolle** wie in anderen industriellen Fertigungsbereichen.
- **Keine Qualitätskontrolle** wie in anderen industriellen Fertigungsbereichen, gerade das Testen ist immer noch unterbewertet.

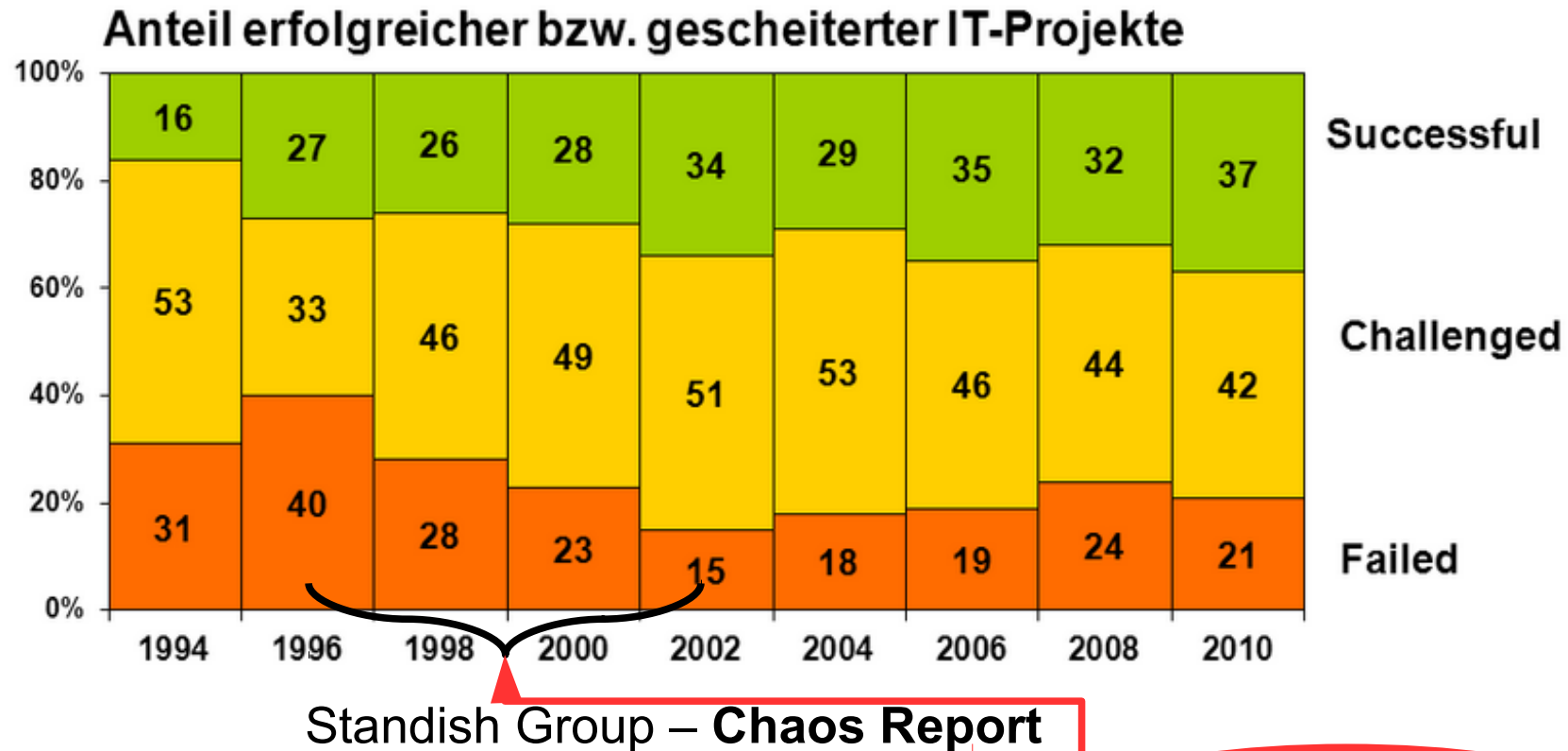
Anzahl Function	Früher als geplant	Termingerecht	Verspätet	Abgebrochen
1 FP	14,86%	83,16%	1,92%	0,25%
10 FP	11,08%	81,25%	5,67%	2,00%
100 FP	6,06%	74,77%	11,83%	7,33%
1.000 FP	1,24%	60,76%	17,67%	20,33%
10.000 FP	0,14%	28,03%	23,83%	48,00%
100.000 FP	0,00%	13,67%	21,33%	65,00%
Durchschnitt	5,53%	56,94%	13,71%	23,82%

Abbruchrate großer Projekte nach [Jon96]

Andere Quelle: **Erfolgreich** durchgeführte **Software-Projekte**:

- 16% im Jahre 1994; 34% im Jahre 2003

1 Function Point (FP): ca. 55 Lines of Code (LOC) in C++/Java, 20 LOC Perl, 13 LOC SQL, etc.
 Desktop-Projekt mit 1 FP dauert ca. eine Woche mit einem Entwickler
 Allgemeines Projekt mit 100.000 FP und 100 Entwicklern: ca. 17 Monate



Erfolgreich durchgeführte **Software-Projekte:**

- 35% seit 15 Jahren

Gescheiterte Software-Projekte:

- 20%

**Abnahme gescheiterter
Projekte: Geänderte
Kriterien**

Interessenkonflikte: Termine, Kosten, Qualität

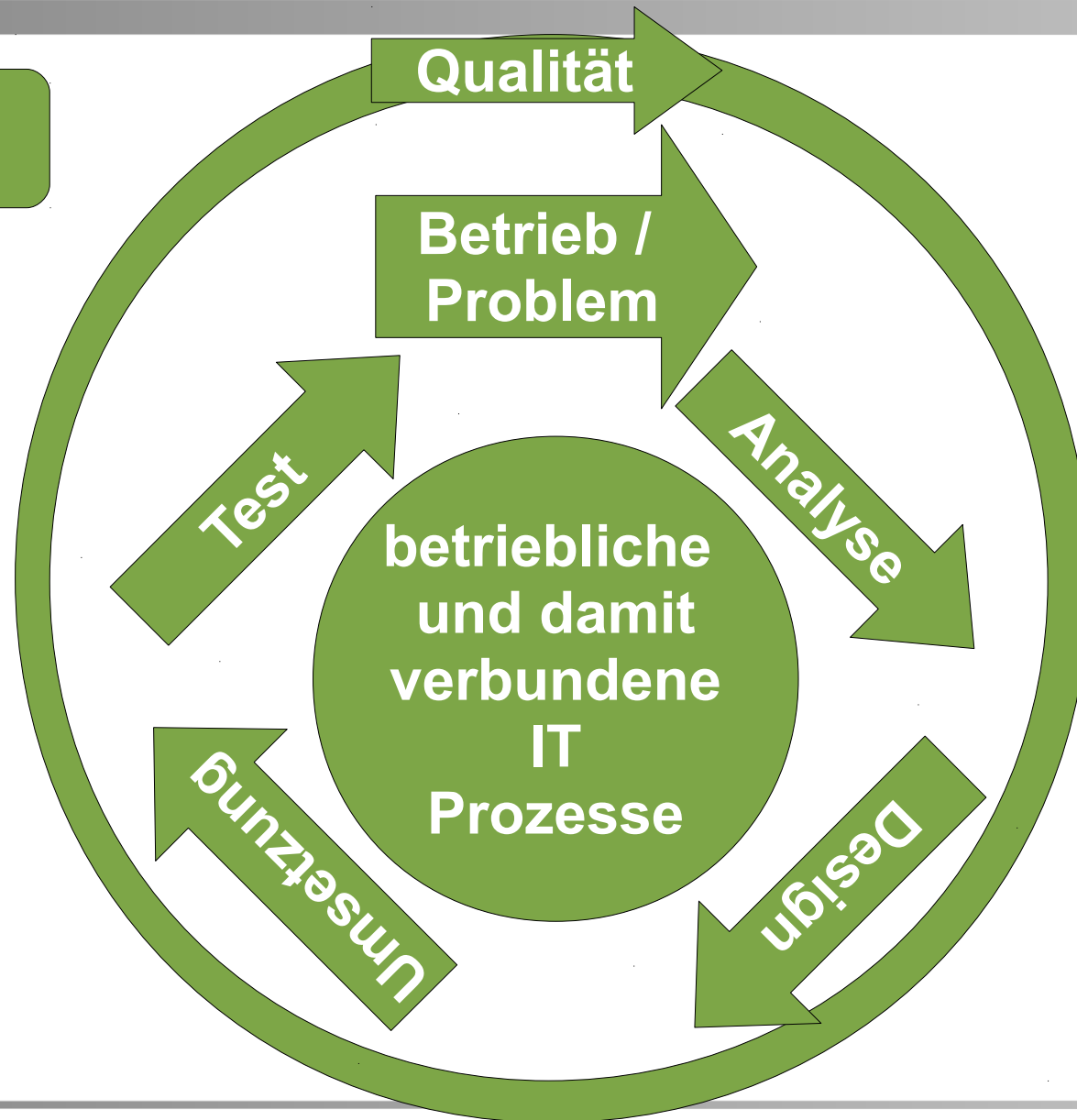
Softwarekonstruktion
WS 2016/17



Der SE Lebenszyklus: Punktuelle Vertiefung



**Umfassend:
Qualität**



Kapitel 1: Modellbasierte Softwareentwicklung

- * Teil 1.0: Modellbasierte Softwareentwicklung: Einführung
- * Teil 1.1: Modellbasierte Softwareentwicklung
- * Teil 1.2: Grundlagen Object Constraint Language (OCL)
- * Teil 1.3: Eclipse Modeling Framework und Metamodellierung
- * Teil 1.4: Petrinetze und nebenläufige Systeme

Kapitel 2: Qualitätsmanagement

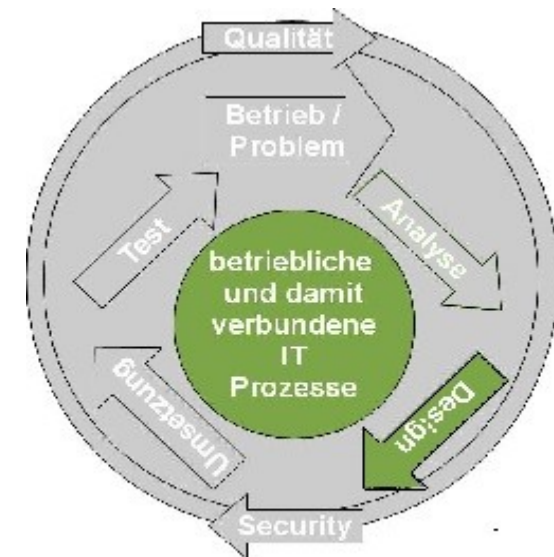
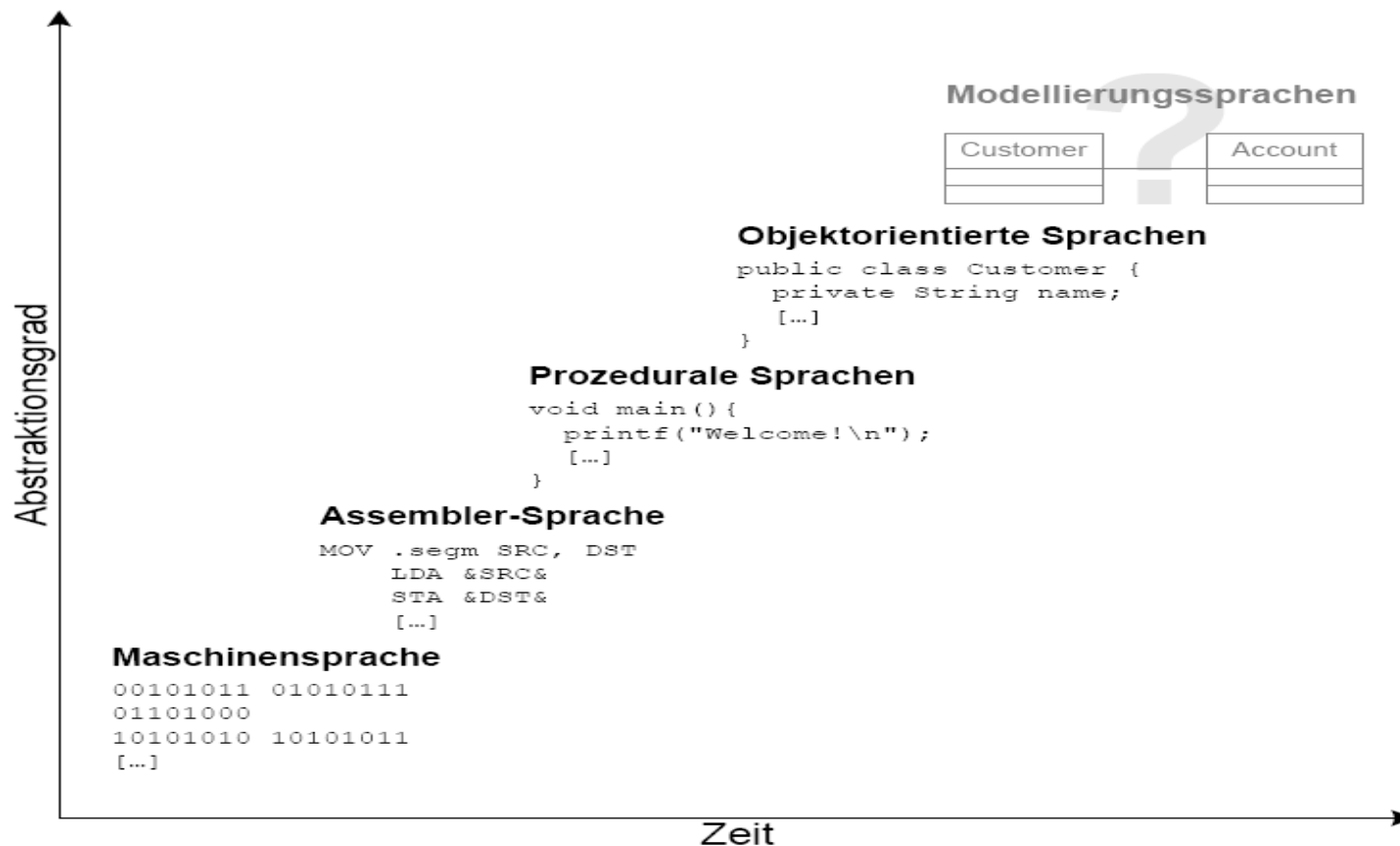
- * Teil 2.0: Einführung Qualitätsmanagement
- * Teil 2.1: Grundlagen der Softwareverifikation
- * Teil 2.2: Softwremetriken und Schätzen
- * Teil 2.3: Black Box Test
- * Teil 2.4: White Box Test
- * Teil 2.5: Testen im Softwarelebenszyklus

Kapitel 3: Fortgeschrittene Softwareentwicklung

- * Teil 3.0: Konfigurationsmanagement
- * Teil 3.1: Softwarearchitektur
- * Teil 3.2: Softwarearchitektur (u.U. Industrievortrag)
- * Teil 3.3: Algebraische Spezifikation
- * Teil 3.4: Einführung in das Projektmanagement

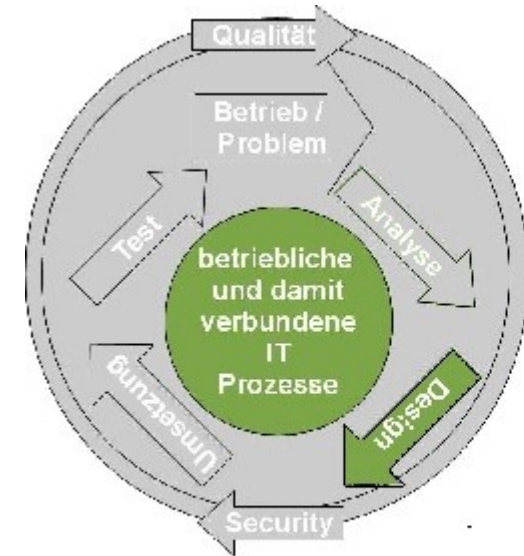
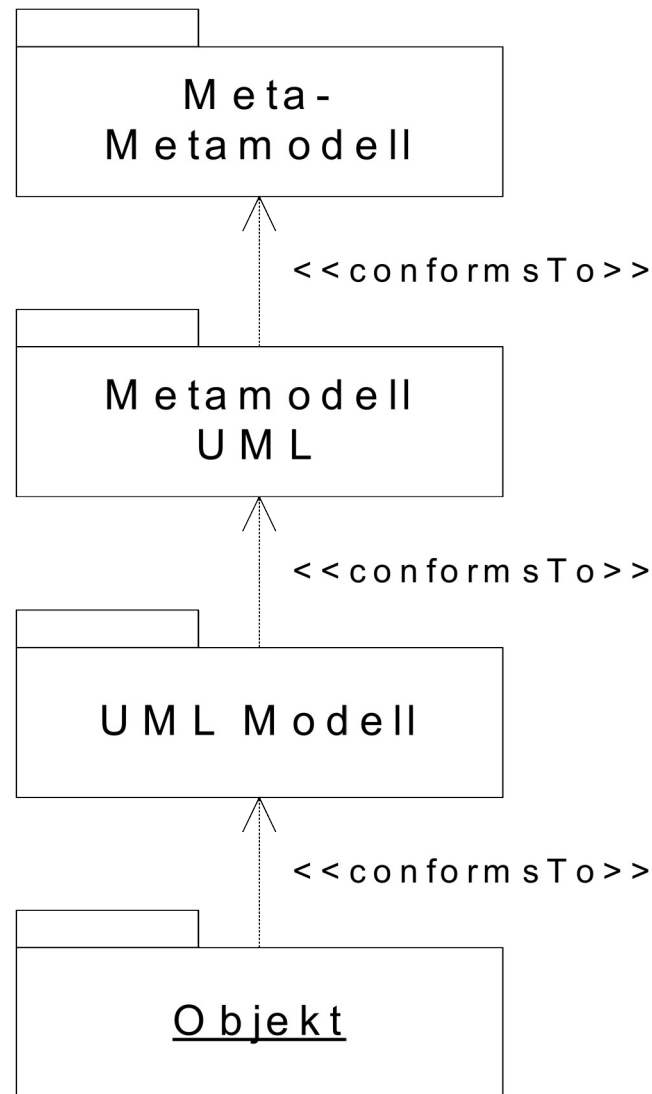
Teil 1.0: Modellgetriebene SW-Entwicklung: Einführung

- Modelle und Modellierung: Begriffe
- UML: Wiederholung
- Szenarien der Modellgetriebenen SW-Entwicklung



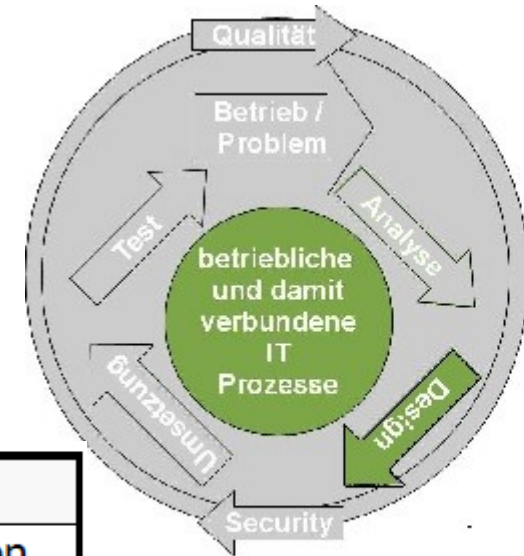
UML schon bekannt aus SWT und SoPra. Hier neu:

- **Metamodellierung**
- **Spezifikation der UML**
- **Model-Driven Architecture**



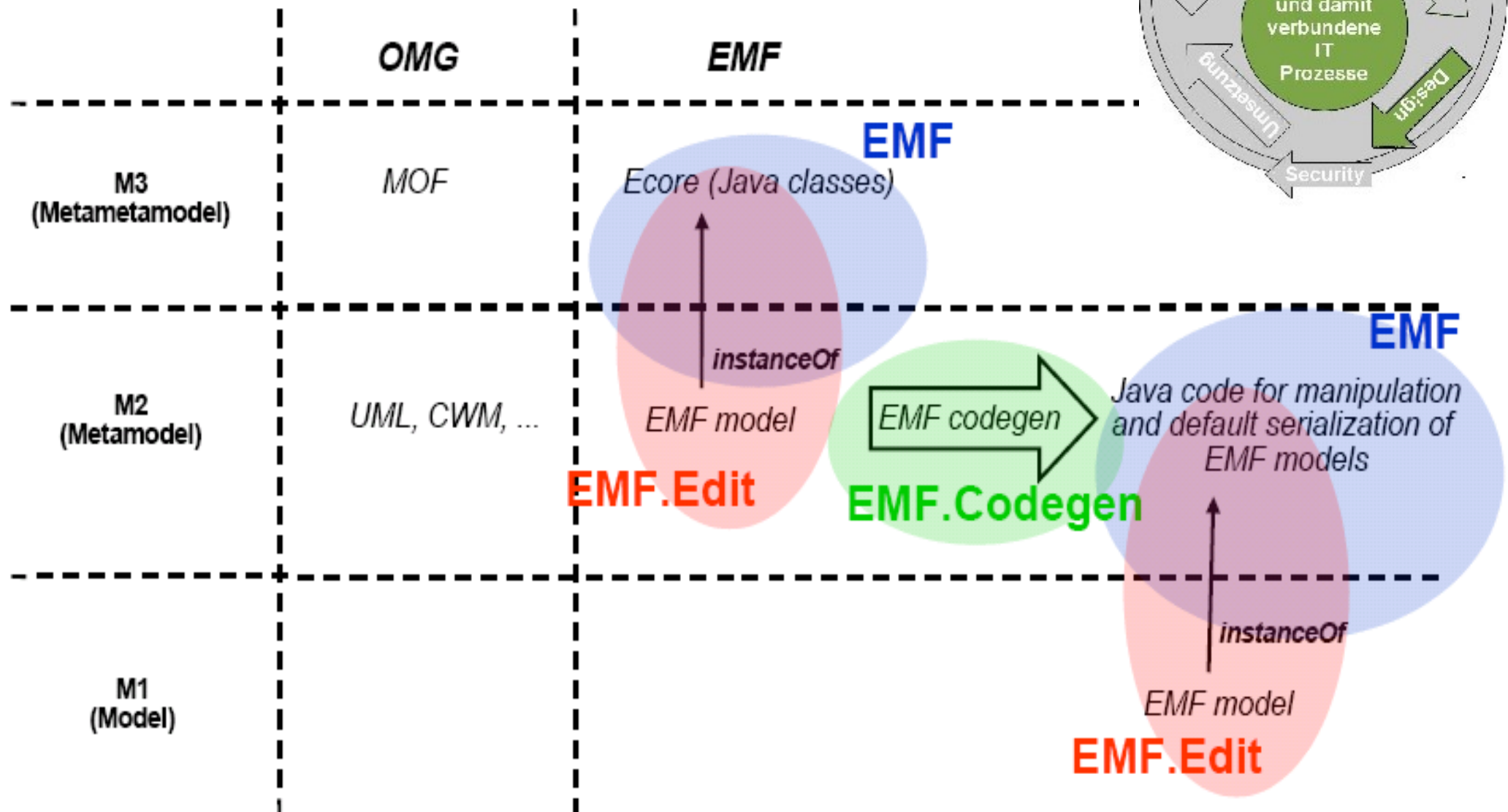
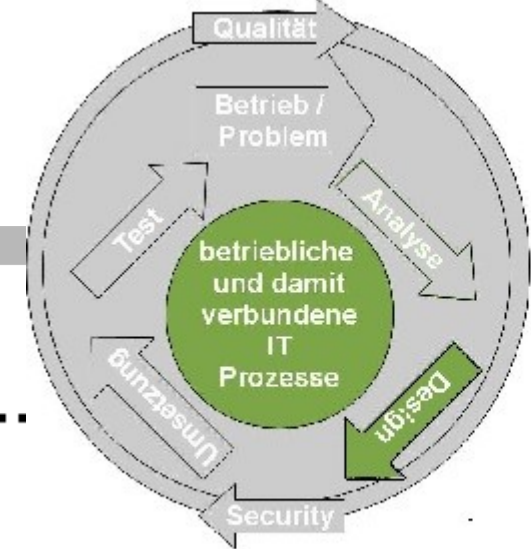
Teil 1.2:

Object Constraint Language (OCL)



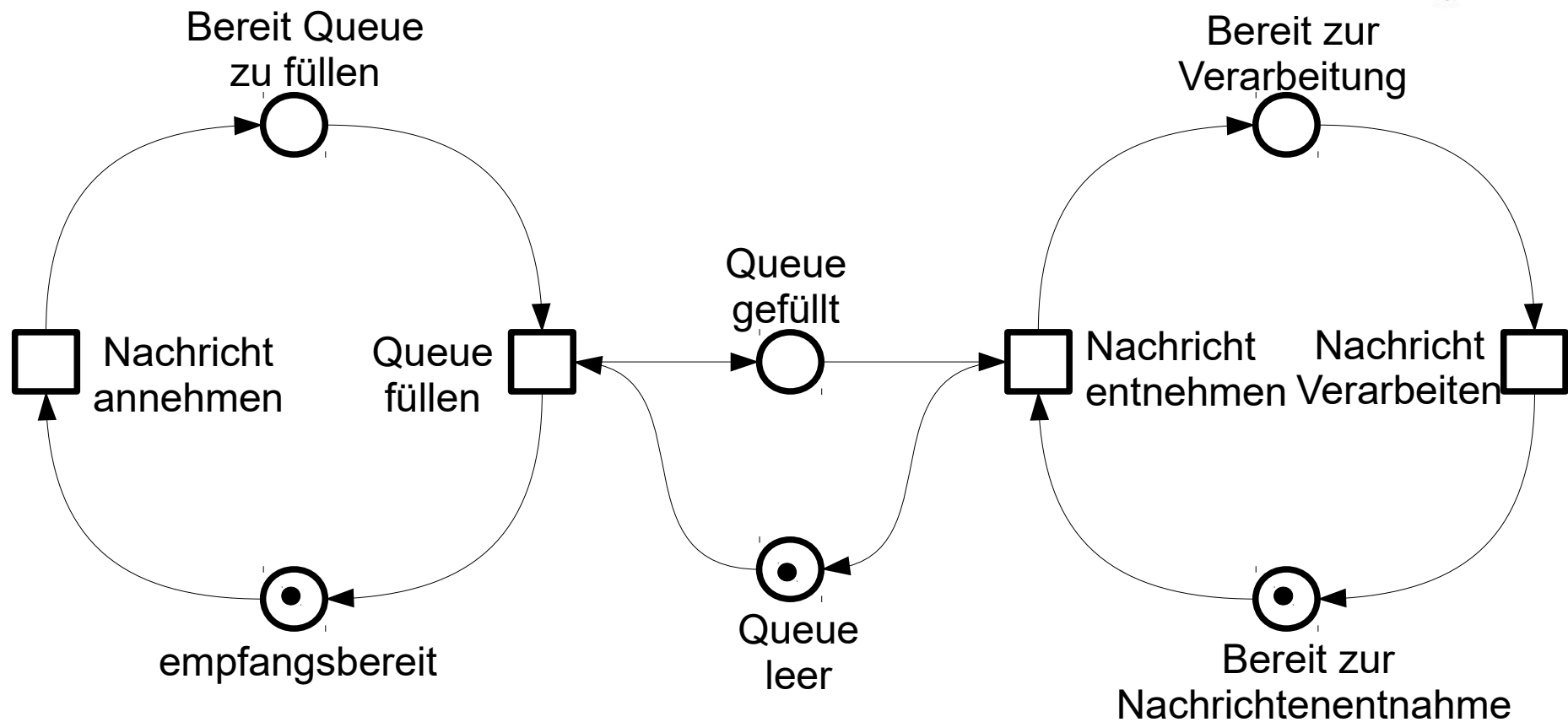
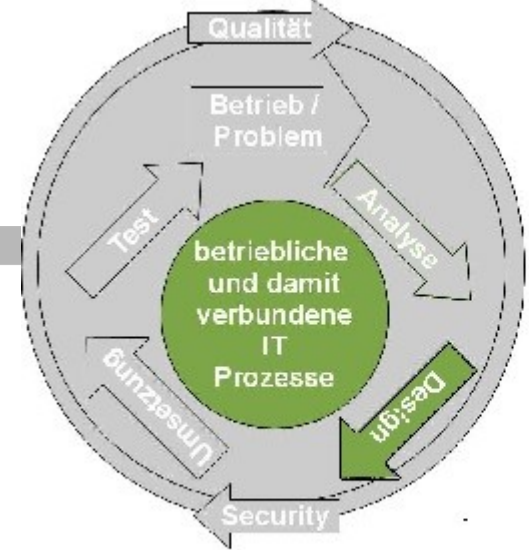
Operation	Description
hasReturned() : Boolean	True if type of template parameter is an operation call, and the called operation has returned a value.
result()	Returns the result of the called operation, if type of template parameter is an operation call, and the called operation has returned a value.
isSignalSent() : Boolean	Returns true if the OclMessage represents the sending of a UML Signal.
isOperationCall() : Boolean	Returns true if the OclMessage represents the sending of a UML Operation call.
parameterName	The value of the message parameter.

Teil 1.3: Eclipse Modeling Framework (EMF)



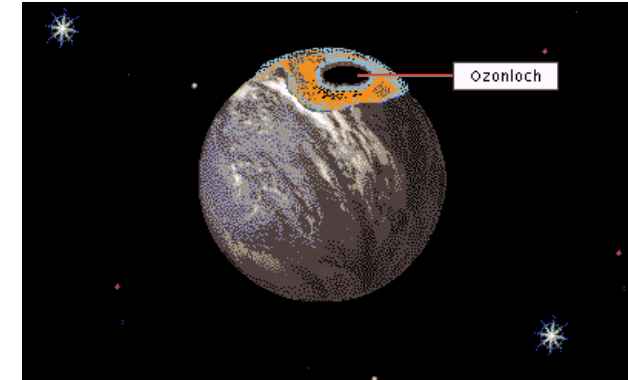
Teil 1.4: Petrinetze

Beispiel: Nachrichten-Queue



Beispiele für Auswirkungen von
Software-Fehlern:

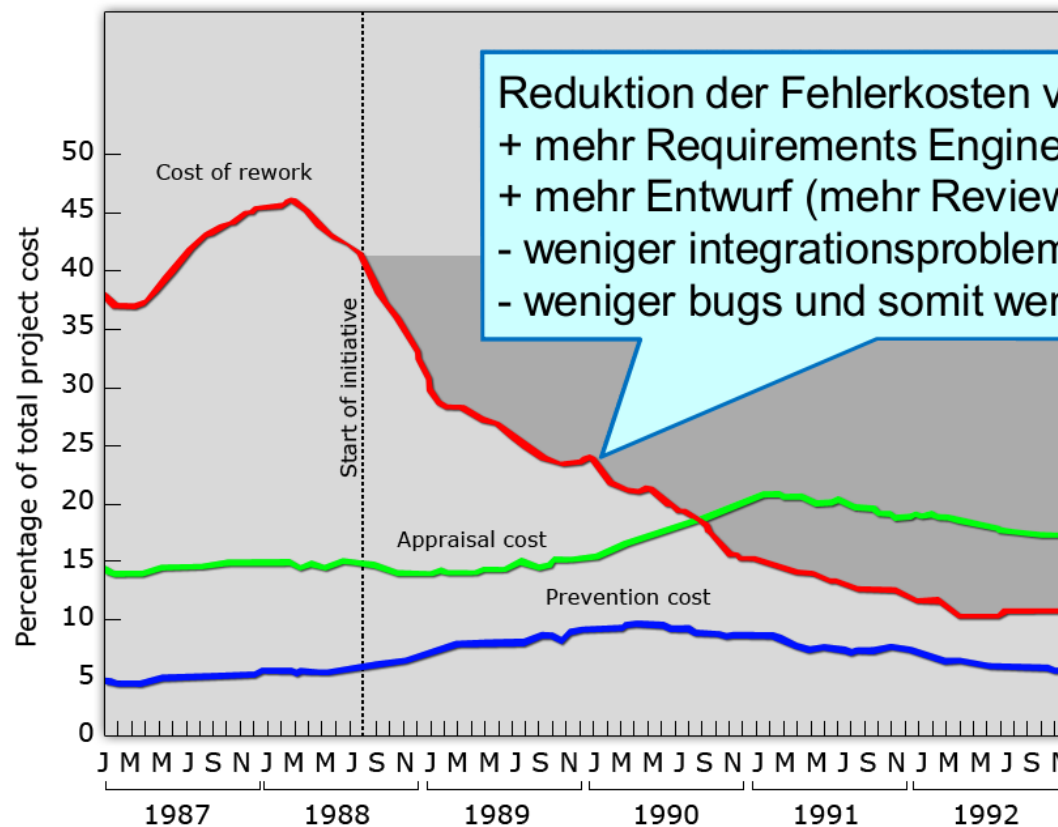
- **NASA - Erdbeobachtungssatelliten**
1979-1985: **Ozonloch** 7 Jahre (!)
lang **nicht erkannt**.
- ESA, Kourou, Franz. Guyana, 4. Juni
1996: **Selbstzerstörung der Ariane**
5 beim Jungfernflug 39 Sekunden
nach Start.
- Bemannte **NASA-Raumkapsel**
Gemini V: **Verfehlte ihren**
Landeplatz um ca. 160 Kilometer.



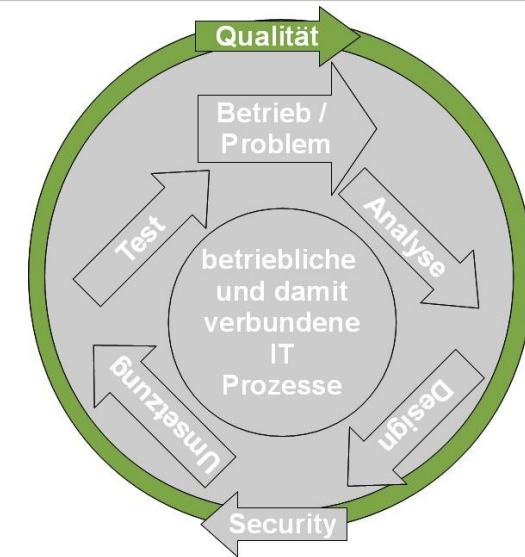
Ariane 5 Flight 501



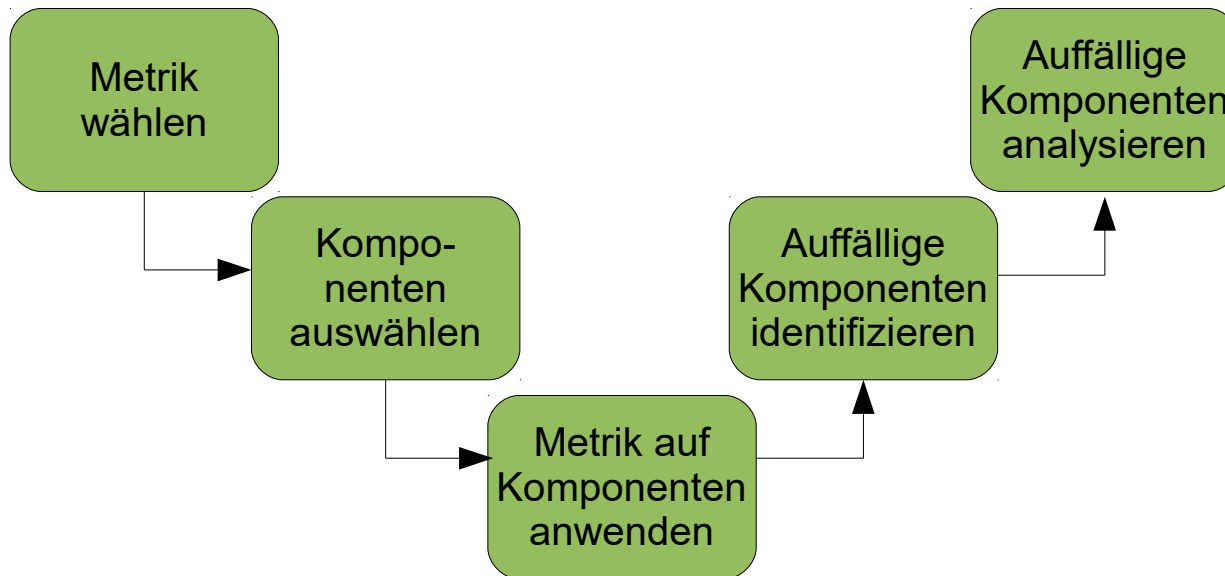
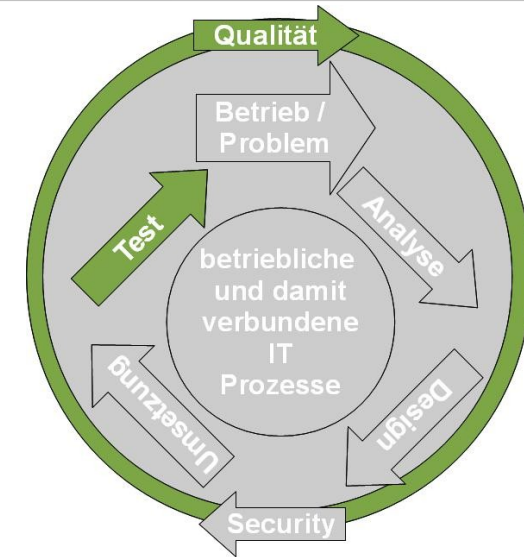
Teil 2.1: Grundlagen der Softwareverifikation



Reduktion der Fehlerkosten von 41% zu 11%:
+ mehr Requirements Engineering
+ mehr Entwurf (mehr Reviews)
- weniger integrationsprobleme mit Sourcecode
- weniger bugs und somit weniger Nachtesten

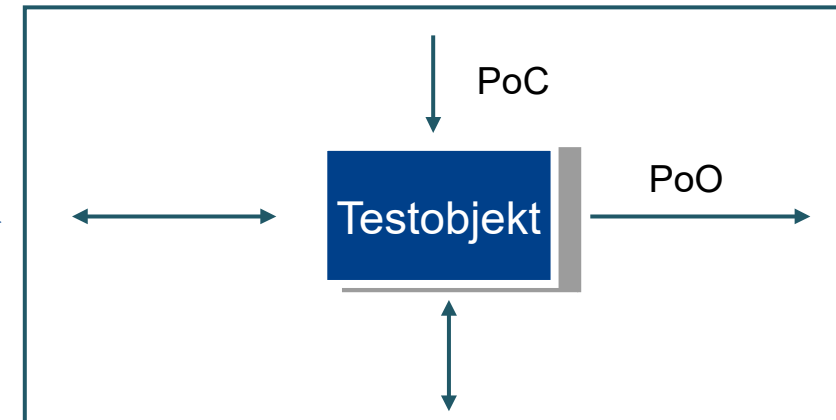
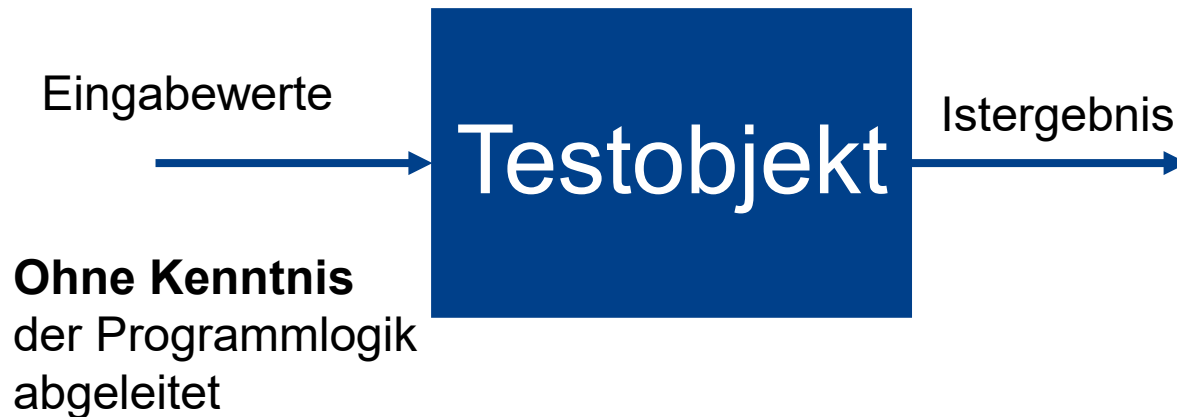
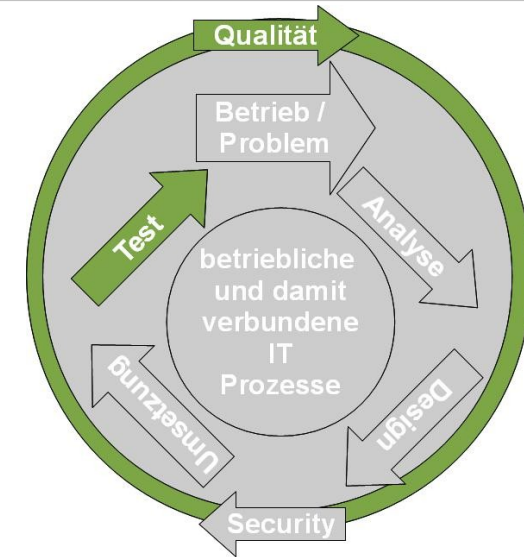


- Metriken
- Direktes und indirektes Messen
- Vorgehensweisen
- Effekte



Äquivalenzklassen schon
bekannt aus SWT. Hier neu:

- **Grenzwertanalyse**
- **zustandbasiertes Testen**

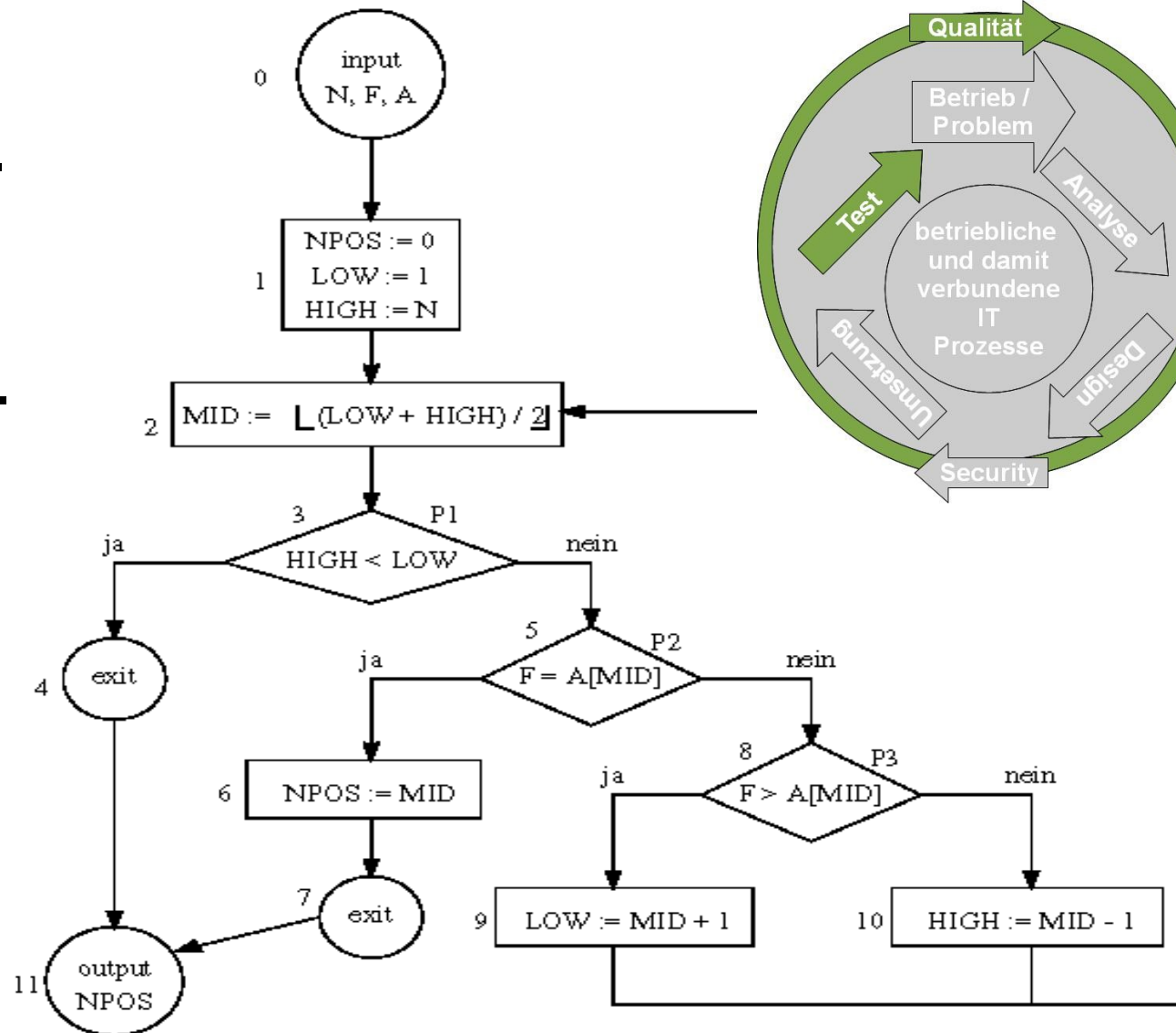


**Daten- & Kontrollfluss-
basiertes Testen.**

**Anweisungs-, Zweig-,
Pfad-, und Mehrfach-
bedingungs-
überdeckung** schon
aus SWT bekannt.

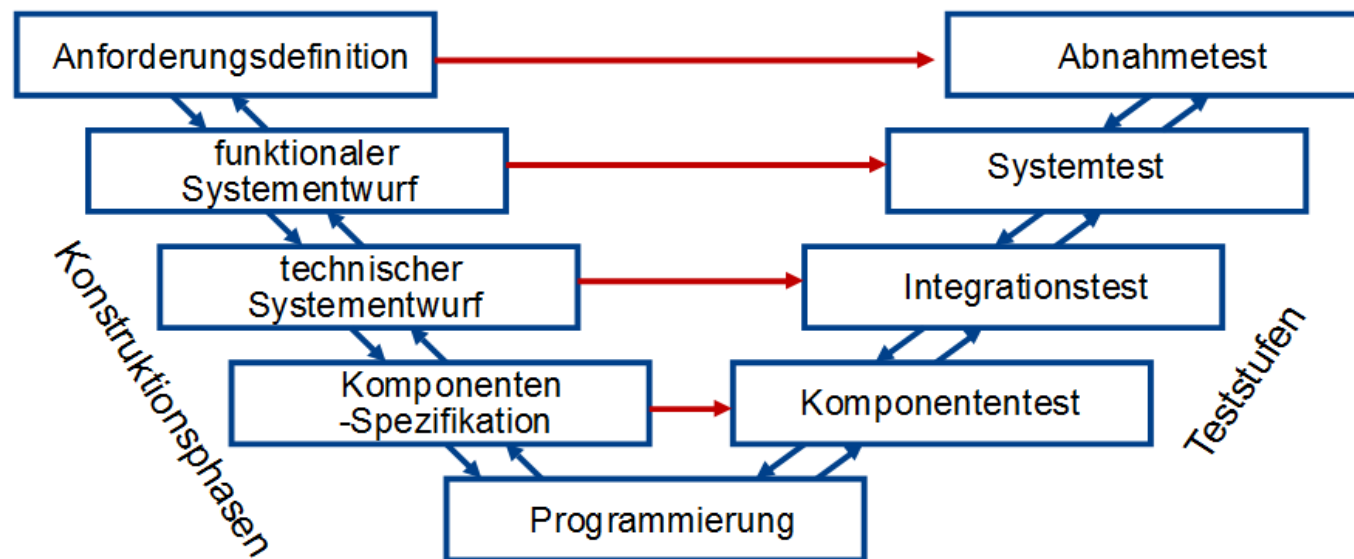
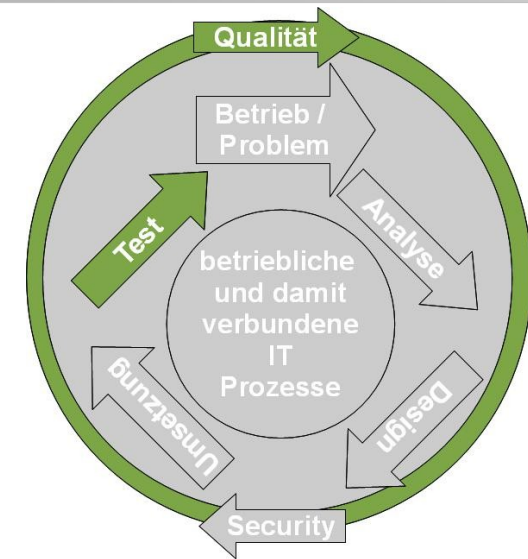
Hier neu:

- datenflussbasierte Testen
- statische Analyse



Programmtest und **Klassentest** schon aus SWT bekannt. Hier neu:

- weitere Testarten (**Komponenten-, Integrations-, System- und Abnahmetest**)
- dazugehörige (**Test-)**Strategien.



Programme zu erstellen, ist bereits aus SWT und SoPra bekannt

- Neu:
 - Wie baut man methodisch grosse Softwaresysteme mit (verteilten) Teams?
 - Wie wird die Qualität des Produkts gesteuert und garantiert?
 - Was macht man, wenn nach der Auslieferung ein Problem vorliegt?

- Entwurfsmuster bekannt aus SWT und SoPra sind Standardlösungen für Probleme bei Objekten
- Neu:
 - Kann ich diese Idee auch verallgemeinern auf Komponenten mit komplexer Funktionalität?
 - Welche Qualität erhalte ich? Wie steuere ich Qualität? Welche Garantien erhalte ich?
 - Was ist eine gute Softwarearchitektur?

- Wie spezifiziere ich Programme mathematisch?
- Welche Programmgarantien erhalte ich aus der Spezifikationen?
- Wie überprüfe ich, ob meine Implementierung die Spezifikation erfüllt?
- Welche Eigenschaften sollte meine Implementierung haben, damit die Spezifikation nicht verletzt wird?

- Wie plane ich, dass Software mit definierter Qualität, Kosten und Zeit erstellt wird? Wiederholt zuverlässig?
- Wie kann ich die Projektkosten abschätzen?
- Wie gehe ich mit Risiken im Projekt um?

Was Sie aus „Softwaretechnik“ und dem „Software-Praktikum“ wissen sollten:

- Qualitätsmerkmale von Software
- V-Modell
 - Testverfahren auf den verschiedenen Ebenen
 - Programmtest, Klassentest, Komponententest, ...
- UML-Diagramme
 - Klassendiagramme
 - Aktivitätsdiagramme
 - ...

- Sommerville, Ian: Software Engineering. Boston: Pearson. ISBN 0-13-705346-0, 9th edition, 2011
- Auch ergänzendes Material unter:
<http://ifs.host.cs.st-andrews.ac.uk/Books/SE9/>

Teil 1.0-1.2:

- J. Ludewig, H. Lichter: **Software Engineering - Grundlagen, Menschen, Prozesse, Techniken**, dpunkt.verlag, 3. Auflage, 2013.
https://www2.swc.rwth-aachen.de/se_buch . **Ab 33 EUR (e-book):**
<http://www.dpunkt.de/buecher/4501/software-engineering.html>
Unibibliothek: <http://www.ub.tu-dortmund.de/katalog/titel/1416968>.
- V. Gruhn, D. Pieper, C. Röttgers: **MDA – Effektives Software-Engineering mit UML2 und Eclipse**, Xpert.press/Springer-Verlag, 2006.
Unibibliothek: <http://www.ub.tu-dortmund.de/katalog/titel/1223129>
- J. Seemann, J.W. Gudenberg: **Software-Entwurf mit UML2**, Xpert.press/Springer-Verlag, 2006. Unibibliothek
<http://www.ub.tu-dortmund.de/katalog/titel/1223020>

(Links s. a. Vorlesungswebseite)

Teil 1.2:

- J. Warmer, A. Kleppe: **The Object Constraint Language: Getting Your Models Ready for MDA**, Addison-Wesley Longman Publ. & Co., Inc., 2003.
Unibibliothek: <http://www.ub.tu-dortmund.de/katalog/titel/901443>
- Object Management Group: **OCML 2.4** <http://www.omg.org/spec/OCML/2.4/PDF/>

Teil 1.3:

- J. Becker, C. Mathas, A. Winkelmann: **Geschäftsprozessmanagement**. Springer-Verl., 2009. Unibib.: <http://www.ub.tu-dortmund.de/katalog/titel/1256897>

Teil 1.4:

- H. Reisig: **Petrinetze**. Vieweg, 2010. Teil I.
Unibibliothek: <http://www.ub.tu-dortmund.de/katalog/titel/1305786>

Teil 1.5:

- D. Steinberg: **Eclipse Modeling Framework**, Addison-Wesley, 2008.

- E. Riedemann: **Testmethoden für sequentielle und nebenläufige Software-Systeme**. Teubner, Stuttgart, 512 S., 1997. PDF herunterladbar von Vorlesungs-Webseite. Unibibliothek:
<http://www.ub.tu-dortmund.de/katalog/titel/687299>
- A. Spillner, T. Linz: **Basiswissen Softwaretest**. 4., überarbeitete Auflage, dpunkt.verlag, 2010, 308 Seiten, 39 Euro (D), ISBN 987-3-89864-642-0
Unibibliothek: <http://www.ub.tu-dortmund.de/katalog/titel/1287855>.
- Bei Engpässen in der Ausleihe alternativ ähnliche Inhalte als e-Book:
A. Spillner, T. Linz, H. Schaefer: **Software Testing Foundations**, 3. Auflage, Rocky Nook, 2011, 296 Seiten, Print ISBN-13: 978-1-933952-78-9
Unibibliothek (**e-Book**):
<http://www.ub.tu-dortmund.de/katalog/titel/1409780>

- **Organisatorisches**
- Studienordnung: Einordnung / Kompetenzen / Struktur / Prüfung
- Vorlesung: Bildungsvertrag, Termine, Feedback
- Übung: Konzept / Termine
- Klausur
- **Vorstellung des Fachgebietes**
- Forschung
- Abschlussarbeiten, Hiwi-Jobs, weiteres Lehrangebot
- **Vorlesungsinhalte**

→ **NOCH FRAGEN ?**